

Joining a Standalone EC2 Node to an EKS Cluster

Cluster: "test-cluster" · Region: us-east-2 · Manual join, outside any node group

A "standalone" node here means a single self-managed EC2 instance launched directly — not through a managed node group, an `eksctl nodeGroup`, or an ASG-backed self-managed node group. It still has to do the same three things any worker node does: run the correct bootstrap process, present an IAM identity the cluster trusts, and reach the control plane over the network.

A node like this has no Auto Scaling Group behind it. If it's ever terminated, nothing replaces it automatically, and it won't participate in rolling node upgrades the way a managed/self-managed node group does. Fine for a one-off or special-purpose node — not a substitute for real node group capacity.

1. IAM node role

Create (or reuse) an IAM role for the instance with these managed policies attached, plus an instance profile wrapping it:

```
aws iam create-role --role-name standalone-node-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{ "Effect": "Allow", "Principal": { "Service": "ec2.amazonaws.com" }, "Action": "sts:AssumeRole" }]
  }'

aws iam attach-role-policy --role-name standalone-node-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy
aws iam attach-role-policy --role-name standalone-node-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
aws iam attach-role-policy --role-name standalone-node-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy

aws iam create-instance-profile --instance-profile-name standalone-node-profile
aws iam add-role-to-instance-profile \
  --instance-profile-name standalone-node-profile \
  --role-name standalone-node-role
```

If the VPC CNI already uses IRSA (a dedicated role for the `aws-node` service account), you can drop `AmazonEKS_CNI_Policy` from the node role — that's the recommended setup, not a requirement.

2. Security group

The instance's security group must allow traffic to/from the cluster's control-plane security group (443 both ways at minimum, plus node-to-node CNI/kubelet ports if running more than one node). Look up what the control plane expects:

```
aws eks describe-cluster --name test-cluster --region us-east-2 \
  --query "cluster.resourcesVpcConfig.clusterSecurityGroupId"
```

Then add a rule on the instance's security group allowing outbound 443 to that security group (and inbound from it, if using private endpoint access).

3. Launch the EC2 instance

Use an EKS-optimized AMI matching the cluster's Kubernetes version, in a subnet belonging to the cluster's VPC, with the instance profile from step 1:

```
aws ssm get-parameter \
  --name /aws/service/eks/optimized-ami/1.31/amazon-linux-2023/x86_64/standard/recommended/image_id \
  --region us-east-2 --query "Parameter.Value" --output text
```

Launch with `aws ec2 run-instances` (or the console) using that AMI ID, the `standalone-node-profile` instance profile, and a subnet from the cluster's VPC.

4. Bootstrap — depends on the AMI family

AL2023 (current default) — uses `nodeadm`

```
cat <<EOF > user-data.yaml
apiVersion: node.eks.aws/v1alpha1
kind: NodeConfig
spec:
  cluster:
```

```
name: test-cluster
apiServerEndpoint: $(aws eks describe-cluster --name test-cluster --region us-east-2 \
--query "cluster.endpoint" --output text)
certificateAuthority: $(aws eks describe-cluster --name test-cluster --region us-east-2 \
--query "cluster.certificateAuthority.data" --output text)
cidr: $(aws eks describe-cluster --name test-cluster --region us-east-2 \
--query "cluster.kubernetesNetworkConfig.serviceIpv4Cidr" --output text)

EOF
```

Pass this as EC2 user data (base64-encoded). `nodeadm` runs it automatically via cloud-init on first boot.

AL2 / Ubuntu / custom AMIs — uses `/etc/eks/bootstrap.sh`

```
/etc/eks/bootstrap.sh test-cluster \
--apiserver-endpoint <endpoint from describe-cluster> \
--b64-cluster-ca <CA data from describe-cluster> \
--kubelet-extra-args '--node-labels=role=standalone'
```

Pass this as user data, or SSH in and run it manually — the same pattern used earlier for the custom Ubuntu 24.04 AMI with `overrideBootstrapCommand`.

5. Register the node's identity with the cluster

This is the step most often missed: IAM alone doesn't create a Kubernetes Node object — the API server also has to be told that this role maps to `system:nodes`. Which mechanism applies depends on the cluster's authentication mode (`CONFIG_MAP` / `API` / `API_AND_CONFIG_MAP`).

Access entries enabled (`API` or `API_AND_CONFIG_MAP`) — the current recommended way

```
aws eks create-access-entry \
--cluster-name test-cluster \
--region us-east-2 \
--principal-arn arn:aws:iam::186579592072:role/standalone-node-role \
--type EC2_LINUX
```

Node-type access entries (`EC2_LINUX` / `EC2_WINDOWS`) can't take `--kubernetes-groups` or an associated access policy — the type itself implies `system:bootstrappers` / `system:nodes`.

Legacy ConfigMap (`CONFIG_MAP` mode)

```
kubectl edit -n kube-system configmap/aws-auth
```

Add:

```
data:
  mapRoles: |
    - rolearn: arn:aws:iam::186579592072:role/standalone-node-role
      username: system:node:{{EC2PrivateDNSName}}
      groups:
        - system:bootstrappers
        - system:nodes
```

6. Verify

```
kubectl get nodes --watch
kubectl describe node <new-node-name>
```

The node should reach `Ready` within a minute or two of the bootstrap script finishing.

If the node never joins

Symptom	Likely cause
Node never appears in <code>kubectl get nodes</code>	IAM role missing from access entries/aws-auth, or wrong ARN (must be the role, not the instance profile)
Instance up, kubelet failing to reach API server	Security group blocking 443 to the control-plane SG, or wrong <code>apiserver-endpoint</code> /CA data in bootstrap args
No outbound connectivity at all	Subnet lacks a route to an IGW/NAT — same checklist as the earlier "EC2 has no internet traffic" troubleshooting
Node joins but shows <code>NotReady</code>	CNI plugin not initializing — check <code>aws-node</code> DaemonSet pod logs on the new node

Sources

- [Create self-managed Amazon Linux nodes — Amazon EKS](#)
- [Maintain nodes yourself with self-managed nodes — Amazon EKS](#)
- [Create access entries — Amazon EKS](#)
- [CreateAccessEntry — Amazon EKS CLI reference](#)
- [Troubleshoot EKS worker nodes that fail to join cluster — AWS re:Post](#)